

FORMS

HTML Forms

CREATED BY: ALEKSANDRA NOVÁK

What is a form?

A **form** is a component of a web page that allows users to interact with it through **form controls** like text fields, buttons, checkboxes, etc., which together form the user interface.

Provided data is then typically sent to a server for processing, such as performing a search or saving user data.

In many cases, forms work without any client-side scripting (JavaScript) as browsers and HTML itself have built-in behaviour for handling input, validation, and form submission.

Form implementation consists of 3 major parts:

- Creating a user interface
- Implementing the server-side processing
- Configuring communication with the server

<form>

The <form> element, similarly to <a>, represents a hyperlink which, instead of leading directly to a set destination, lets firstly enter or change some data (**editable form-associated elements**) that will then be sent or used when the form is submitted.

Form is manipulated through **form-associated elements** (ones that can have a form owner). The most important ones:

`button`, `fieldset`, `input`, `output`, `select`, `textarea`, `img`

Labelable elements

A group of elements that can be associated with **<label>**.

The label should be present if, for example, the element is editable (select/textarea/input), and the user should know what it represents.

Label is not necessary when, for example, the context is obvious, like a button has its own “submit” text, or when the element is not interactive, as its purpose is more related to visualisation.

The most important labelable elements:

`button`, `input` (if the `type` attribute is *not* in the `Hidden` state), `meter`, `output`, `progress`, `select`, `textarea`

Label binding

1.for +id

```
<label for="user-email">Email address:</label>
<input id="user-email" name="user_email" type="email">
```

Attribute **for** specifies for which element this label is for. Since each element can be uniquely identified by its **id**, the for attribute references that id.

The **name** attribute in a form control element **does not have to match the id**. Typically, it is sent to the server and processed by the backend. Having the same name is usually convenient.

2. Nesting

The above is the most generic way of bading label to input. Another way would be nesting input inside label; however, this method is limited:

<label> Label: <input> </label>

What does our backend get?

When a form is successfully submitted, the browser serialises all form controls (those with a name and not disabled) into a series of **name–value pairs,** except:

- **disabled** elements
- elements without a **name** attribute
- not-checked checkbox and radio

Value may be specified by the **value** attribute and, in the case of text-related inputs, overridden by the user. For text-related inputs, even though there is a value without a name, it will not be sent. Name without a value will be sent.

```
<label> Name: <input type="text" name="name" value="John"></label>
<label> <input type="radio" name="gift" value="received" checked> Received </label>
```

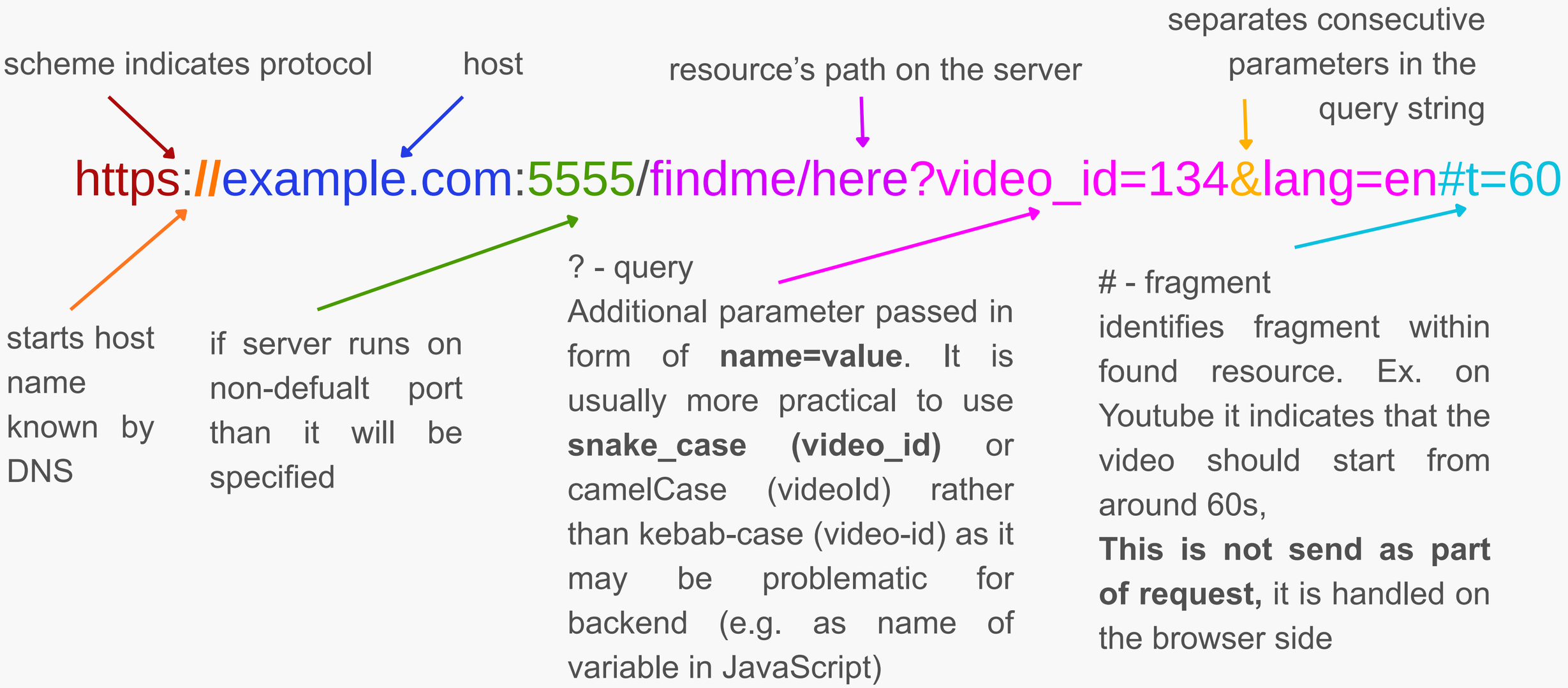
Assuming default GET method and not specified action, data will be set to the same URL as the current document:

```
http://127.0.0.1:3002/task1.html?name=John&gift=received
```

Attributes	**Type/Value**	**Default**
action	URL	Current webpage address
method	GET - retrieving data POST - sending data to server	GET
enctype	for POST: • **Without sending files**: application/x-www-form-urlencoded • **With sending files**: multipart/form-data	application/x-www-form-urlencoded
accept-charset	e.g. UTF-8	inherited from the document
autocomplete	on/off	on

URL

Subtype of URI (Uniform Resource Identifier, it identifies resource), which identifies resource by its location or by the way it can be accessed



Naming Convensions

There is no one recommended naming convention when it comes to HTML attributes. However, due to a specific purpose, the following should hold:

- **id, class, endpoint**: as it will be used in CSS and, in the case of id, eventually in JS, the **kebab-case** is more suitable, e.g. product-name

- **name:** using “-” is could problematic when it comes to backend, so any value that is sent there should not be in kebab-case. Choosing other conventions can be beckend technology depended if one is known. For **JavaScript**, it is **camelCase** (e.g. productName), whereas for **Django,** it is **snake_case** (e.g. product_name).

TASK 1

Build a pure-HTML form. This task focuses on editable elements, so for now, use the <form> tag without attributes.

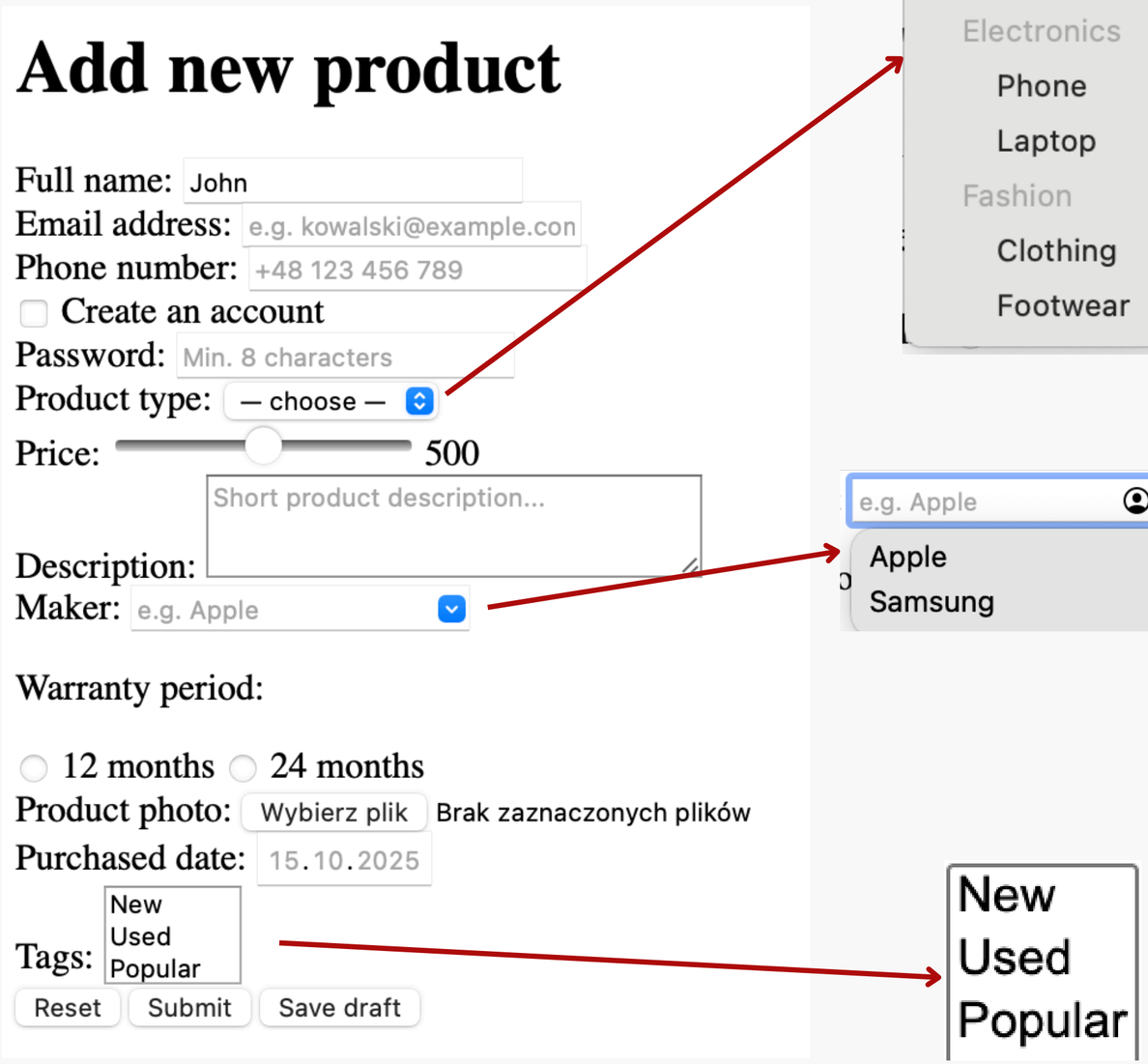
Use:

- input for name, email, phone number, password, purchase date, with a correct type
- checkbox “create account”
- select with optgroup for product type,
- range + output for price
- textarea for description,
- radio for 12/24 months warranty,
- file upload for product photo
- select (multiple) for tags
- datalist for maker

Put each label+editable element in its own . Connect labels via for and id (use kebab-case for id, snake_case/camelCase for name). Add Reset and Submit buttons.

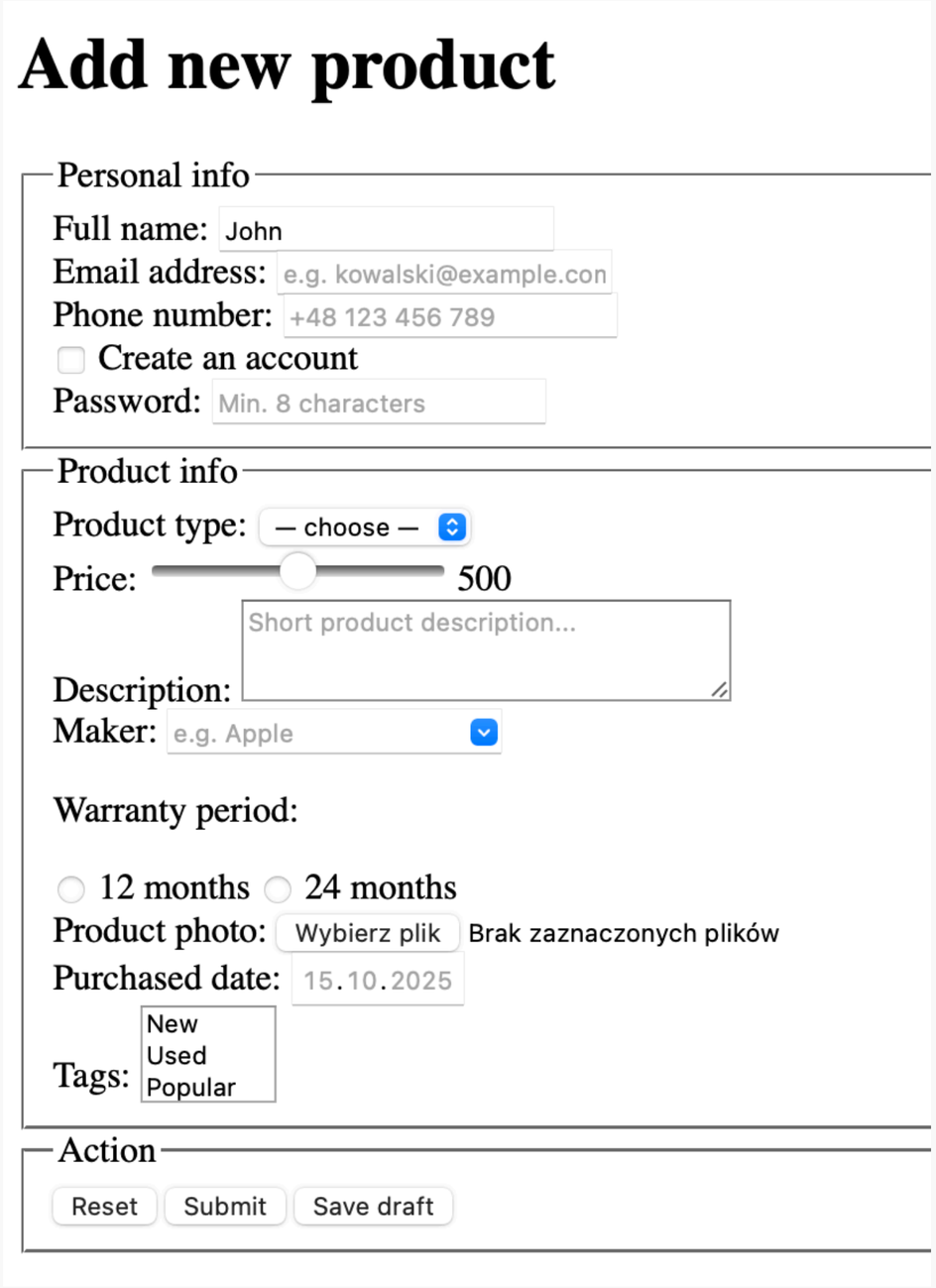
Add:

- placeholders
- min and max length to text-related inputs
- size to select
- rows and cols to textarea
- limit date picker
- add pattern (regex) e.g. password ^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{8,}$



TASK 2

Specify attributes for the <form> tag so that the data can be correctly sent (post method with matching enctype). Add **<fieldset>** and **<legend>** where suitable.



TASK 3

Add and test boolean attributes to the solution from task 1:

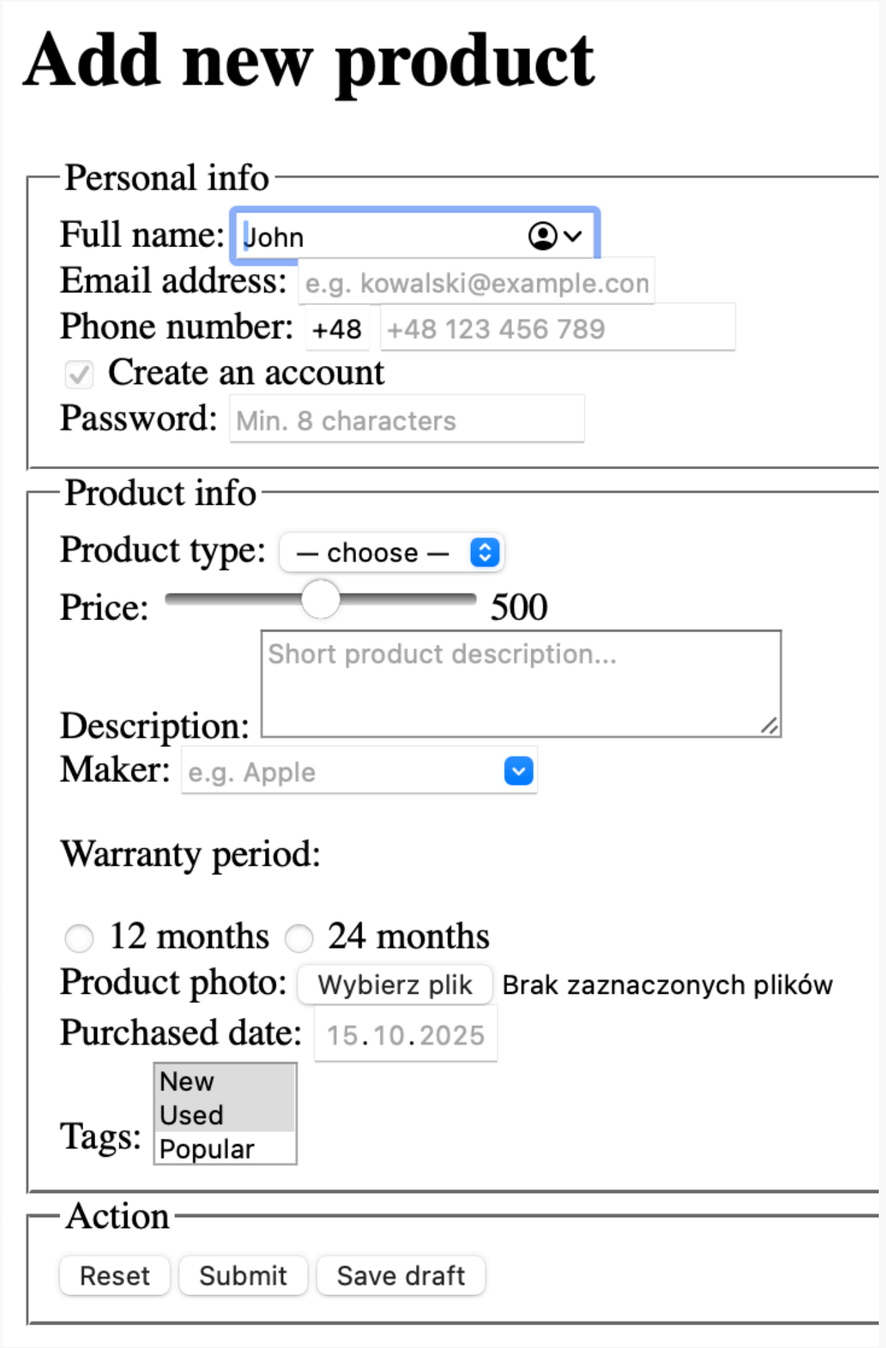
- Full name: **autofocus**
- Email**: required, disabled + required**
- Create an account: **checked, checked + disabled**
- Product type (-choose- opt.)**: disabled selected hidden**
- Warranty period**: checked** for one and both inputs
- Tags: **multiple**
- Phone Number: add a **readonly** input for dial code
- Password**: title, pattern + title** (eventually required for easier testing)

Test validation boolean attributes:

- Save draft: **formnovalidate**
- <form>: **novalidate**

Autocomplete enumerated attribute:

- <form> : change to **off**
- Personal info fields: override only for some fields



Disclaimer

- Tasks 1-3 were adapted from the course materials “HTML - part II”, tasks 1-4, of the Web Application Programming/Programowanie Aplikacji WWW course by dr hab. inż. prof. uczelni Maciej Grzenda (PhD at the time of creation), website: http://www.mini.pw.edu.pl/~grzendam .

- The content presented in the presentation is consistent with the above-mentioned materials.

- The content is primarily based on the HTML Living Standard documentation referenced on each slide, unless stated otherwise.

- All materials are provided for educational and non-commercial use only.